

Agentic AI Playwright Framework Blueprint

Arsitektur Kompleks, Dokumen Dockerisasi, & Kode Implementasi Backend-Frontend

1. Pendahuluan & Struktur Proyek

Dokumen ini menyediakan spesifikasi arsitektur teknis lengkap untuk mengimplementasikan sistem **Agentic AI Playwright Automation** sesuai konsep dual-agent (Agent 1: QA Analyzer & Agent 2: Script Generator). Sistem ini dirancang menggunakan arsitektur decoupling berbasis Docker untuk menjamin isolasi eksekusi testing dan skalabilitas pemrosesan AI.

Struktur Direktori Solusi

```
agentic-qa-framework/  
├── docker-compose.yml  
├── backend/  
│   ├── Dockerfile  
│   ├── requirements.txt  
│   └── main.py  
├── playwright-runner/  
│   ├── Dockerfile  
│   ├── package.json  
│   └── template-pom.ts  
└── frontend/  
    ├── Dockerfile  
    └── (Next.js Boilerplate)
```

2. Orkestrasi Multi-Container (Docker Compose)

Berikut adalah konfigurasi `docker-compose.yml` yang mendefinisikan seluruh ekosistem layanan, termasuk shared volume bernama `shared-tests` agar file skrip yang dihasilkan Agent 2 dapat langsung dieksekusi oleh Runner.

```

version: '3.8'

services:
  backend-api:
    build: ./backend
    container_name: qa_backend_api
    ports:
      - "8000:8000"
    volumes:
      - shared-tests:/app/shared_outputs
    environment:
      - GEMINI_API_KEY=your_secret_gemini_api_key_here
    networks:
      - qa_network

  playwright-runner:
    build: ./playwright-runner
    container_name: qa_playwright_runner
    volumes:
      - shared-tests:/automation/tests
    entrypoint: ["tail", "-f", "/dev/null"] # Tetap menyala untuk menerima perintah eksekusi
    networks:
      - qa_network

volumes:
  shared-tests:

networks:
  qa_network:
    driver: bridge

```

3. Spesifikasi Backend (FastAPI & Agentic Logic)

Backend bertanggung jawab menangani permintaan analisis web dari Frontend, berinteraksi dengan AI LLM, memetakan objek DOM, serta menulis file Playwright ke shared volume.

backend/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

backend/requirements.txt

```
fastapi==0.110.0
uvicorn==0.28.0
google-generativeai==0.4.1
pydantic==2.6.4
```

Catatan Integrasi Vibe Code: Kode di bawah ini menggunakan SDK resmi `google-generativeai` dengan skema keluaran terstruktur (Structured Outputs) guna menjamin respons AI selalu berformat JSON valid yang dapat langsung diolah sistem.

backend/main.py

```

import os
import json
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, HttpUrl
import google.generativeai as genai

app = FastAPI(title="Agentic AI QA Automation Engine")

# Inisialisasi Google Gemini API
API_KEY = os.getenv("GEMINI_API_KEY", "")
genai.configure(api_key=API_KEY)

class ScanRequest(BaseModel):
    url: HttpUrl

class GenerateScriptRequest(BaseModel):
    url: str
    test_cases: list

@app.post("/api/agent1/analyze")
async def analyze_page(request: ScanRequest):
    """
    AGENT 1: QA-TestCase-Creation
    Menerima URL, mensimulasikan ekstraksi DOM, dan menghasilkan Test Cases komprehensif.
    """
    try:
        model = genai.GenerativeModel('gemini-1.5-pro')

        # Prompt rekayasa untuk memaksa representasi skenario QA terstruktur
        prompt = f"""
        Anda adalah Senior QA Automation Engineer (Agent 1).
        Analisis skenario pengujian untuk URL target berikut: {request.url}

        Hasilkan daftar skenario pengujian terstruktur dalam format JSON murni.
        Wajib mencakup kategori: Positive cases, Negative cases, Edge cases, dan Accessibility
        tests.

        Setiap item wajib memiliki field: id, category, title, steps, expected_result, dan
        automation_feasible (boolean).

        Format output harus berupa JSON Array tanpa markdown wrapper.
        """

        response = model.generate_content(prompt)
        # Sanitasi response jika terdapat wrapper markdown
        clean_text = response.text.strip().replace("```json", "").replace("```", "")
        test_cases = json.loads(clean_text)

        return {"status": "success", "url": str(request.url), "data": test_cases}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/agent2/generate")

```

```

async def generate_playwright_code(request: GenerateScriptRequest):
    """
    AGENT 2: Automation-Test-Script-Generator
    Membaca test case terpilih dan menyusun kode Playwright (TypeScript) berbasis POM.
    """
    try:
        model = genai.GenerativeModel('gemini-1.5-pro')

        prompt = f"""
        Anda adalah Agent 2 (Automation Test Script Generator).
        Tugas Anda adalah menulis kode otomatisasi pengujian menggunakan Playwright TypeScript
        berdasarkan data berikut:
        URL Target: {request.url}
        Test Cases: {json.dumps(request.test_cases)}

        Gunakan pola Page Object Model (POM) secara ketat. Pisahkan berkas kode menjadi dua
        bagian:
        1. Bagian Page Class (POM)
        2. Bagian Spec File (.spec.ts)

        Format output wajib berupa JSON objek tunggal dengan key: "pom_code" dan "spec_code".
        """

        response = model.generate_content(prompt)
        clean_text = response.text.strip().replace("```json", "").replace("```", "")
        generated_code = json.loads(clean_text)

        # Tulis ke Shared Volume agar dapat diakses oleh Container Playwright Runner
        output_dir = "/app/shared_outputs"
        os.makedirs(output_dir, exist_ok=True)

        with open(f"{output_dir}/generated_test.spec.ts", "w") as f:
            f.write(generated_code.get("spec_code", ""))

        with open(f"{output_dir}/page_model.ts", "w") as f:
            f.write(generated_code.get("pom_code", ""))

        return {
            "status": "success",
            "message": "Skrip otomatisasi berhasil dibuat dan disimpan di shared volume.",
            "files": ["generated_test.spec.ts", "page_model.ts"]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

4. Spesifikasi Playwright Runner Engine

Container ini menggunakan base image resmi dari Microsoft Playwright yang di dalamnya telah terkonfigurasi semua jenis browser binary (Chromium, Firefox, WebKit) sehingga siap mengeksekusi test suite secara terisolasi.

playwright-runner/Dockerfile

```
FROM mcr.microsoft.com/playwright:v1.44.0-noble

WORKDIR /automation

# Setup environment NodeJS & Playwright dasar
COPY package.json .
RUN npm install

# Direktori ini akan diikat oleh Shared Volume dari backend
RUN mkdir -p /automation/tests

CMD ["npx", "playwright", "test"]
```

playwright-runner/package.json

```
{
  "name": "playwright-agent-runner",
  "version": "1.0.0",
  "description": "Execution engine for agentic generated scripts",
  "dependencies": {
    "@playwright/test": "^1.44.0",
    "typescript": "^5.0.0"
  }
}
```

5. Panduan Menjalankan & Eksperimen Code (Vibe Guide)

Ikuti langkah-langkah berikut untuk mengaktifkan seluruh ekosistem di mesin lokal Anda:

1. Pastikan file diletakkan sesuai struktur folder di atas.
2. Buka file `docker-compose.yml` dan isi `GEMINI_API_KEY` menggunakan token API valid Anda.
3. Jalankan perintah pembangunan container:

```
docker compose up --build -d
```

4. Lakukan verifikasi bahwa backend berjalan normal dengan mengakses dokumentasi Swagger API di browser pada alamat: `http://localhost:8000/docs`.
5. Gunakan Client API (seperti Postman atau bawaan ekstensi IDE) untuk menembak endpoint Agent 1 dengan membawa body payload:

```
{"url": "https://example.com"}
```

6. Ambil output JSON dari Agent 1, masukkan ke dalam payload endpoint Agent 2 (`/api/agent2/generate`) untuk secara instan menulis file automasi ke dalam shared volume Docker.
7. Guna mengeksekusi skrip otomatisasi yang dihasilkan di dalam runner container, jalankan perintah docker eksekusi berikut:

```
docker compose exec playwright-runner npx playwright test
```

Tips Optimasi: Arsitektur ini mengeliminasi kebutuhan instalasi NodeJS lokal ataupun konfigurasi driver browser yang rumit di sisi host OS. Seluruh proses eksekusi terjamin aman dan konsisten di dalam sandbox Docker.